

Análisis de la Seguridad del Protocolo MTPROTO de Telegram

Areli Alejandra Guevara Badillo
aguevarab1900@alumno.ipn.mx

Ricardo Mora Santillán
rmoras1900@alumno.ipn.mx

Resumen— Telegram es una aplicación de mensajería que combina sincronización en la nube con cifrado de extremo a extremo en sus chats secretos. Utiliza el protocolo propio MTPROTO, basado en primitivas clásicas como AES-256, RSA-2048, SHA-1 y Diffie-Hellman, con el fin de garantizar confidencialidad y autenticación. Sin embargo, su diseño ha recibido críticas por vulnerabilidades que lo alejan de los estándares modernos de cifrado autenticado, situándolo en un punto intermedio entre seguridad y eficiencia para entornos móviles.

Palabras clave— Mensajería instantánea, seguridad de la información, Telegram, cifrado, MTPROTO

I. INTRODUCCIÓN

En la actualidad, las aplicaciones de mensajería instantánea representan un pilar fundamental en la comunicación digital al ser utilizada por millones de usuarios alrededor del mundo, y con ello miles de datos diariamente que se deben proteger para evitar un mal manejo de ellos. Entre las diversas plataformas podemos encontrar a Telegram, la cual fue lanzada en 2013 por los hermanos Dúrov posicionándose como una de las aplicaciones más utilizadas gracias a su fácil manejo, rapidez y seguridad [1].

A diferencia de otras plataformas o servicio de mensajería, esta cuenta con sus propios mecanismos de cifrado y sincronización en la nube, la cual, sumado a su mecanismo de chats cifrados de extremo a extremo la convierten en un excelente caso de estudio para las mensajerías instantáneas.

II. TELEGRAM Y LA SEGURIDAD DE LA INFORMACIÓN

A. ¿Qué es Telegram y cómo funciona?

Telegram es una aplicación de mensajería enfocada en la velocidad y seguridad, con una sincronización constante con la nube permitiendo acceder a las conversaciones en distintos dispositivos al mismo tiempo, así mismo cuenta con la capacidad para ofrecer llamadas de voz, videollamadas o incluso enviar archivos de hasta 2GB (actualmente esto se encuentra ampliado), esto mediante el uso de un cifrado end-to-end, lo que le permite ser un servicio de mensajería que satisfaga todas las necesidades personales o de negocios[1].

Para brindar una mayor seguridad se emplean un chat secreto en el cual los dispositivos participantes intercambian claves de cifrado o también conocidas como claves de intercambio Diffie-Hellman, para posteriormente proceder a

la conexión segura end-to-end. Así mismo manejan un sistema de procesamiento donde los mensajes y datos son procesados únicamente por el chat del destinatario, almacenando y procesando secuencias aleatorias de símbolos que no tienen significado a través de los servidores y la nube, manteniendo de esta forma un cifrado servidor-cliente[1][2].

B. Diseño inicial vs actual

En su concepto original, Telegram fue diseñado como una aplicación de mensajería instantánea que buscaba enlazar la velocidad, confiabilidad y seguridad. Para alcanzar este objetivo, sus desarrolladores decidieron en vez de utilizar construcciones criptográficas probadas, optaron por crear su propio protocolo original para cifrado simétrico, conocido como MTPROTO (Telegram Messenger Protocol). La motivación principal fue optimizar el rendimiento en dispositivos móviles y redes con anchos de banda limitados, garantizando simultáneamente la transmisión de archivos de gran tamaño.

El diseño inicial de MTPROTO se fundamentaba en el uso de AES-256 en modo Infinite Garble Extension (IGE) como esquema de cifrado simétrico, complementado con el intercambio de claves Diffie-Hellman para establecer claves secretas compartidas entre los extremos de la comunicación. Asimismo, se empleaba RSA-2048 en el proceso de autenticación y SHA-1 como función hash para la verificación de integridad. Sobre esta base, Telegram ofrecía dos modalidades de comunicación diferenciadas:

1. **Chats en la nube**, con cifrado cliente-servidor, que permitían la sincronización de mensajes entre múltiples dispositivos del usuario.
2. **Chats secretos**, que implementaban cifrado de extremo a extremo, garantizando que únicamente los participantes tuvieran acceso al contenido del intercambio.

La decisión de Telegram de desarrollar un protocolo propio, MTPROTO, en lugar de adoptar un estándar existente, se basa en una filosofía de diseño particular centrada en la eficiencia para dispositivos móviles.

Según los propios desarrolladores, su enfoque se justifica en la elección de algoritmos probados a lo largo del tiempo. Ellos prefieren hacer uso de algoritmos conocidos, creados en la época en que el ancho de banda y la potencia de procesamiento eran productos mucho más escasos, argumentando que tiene

efectos secundarios valiosos para el desarrollo móvil moderno y el envío de archivos grandes, siempre que se solucionen los inconvenientes conocidos. Reconocen que las debilidades de estos algoritmos también son bien conocidas y han sido explotadas durante décadas, pero afirman que los utilizan en una combinación tal que, hasta donde sabemos, evita cualquier ataque conocido[1].

No obstante, este diseño inicial recibió críticas desde la comunidad académica y de seguridad informática. Diversos estudios evidenciaron que MTPROTO no cumplía con propiedades modernas de seguridad, como la indistinguibilidad bajo ataque de texto cifrado elegido (IND-CCA) [3].

Se identificaron vulnerabilidades en el manejo del relleno (padding) y en los mecanismos de verificación de integridad, lo que abría la posibilidad de realizar ataques activos mediante la manipulación de los mensajes sin ser detectados (véase sección C. Problemas y vulnerabilidades).

Ante tales observaciones, el diseño actual de Telegram ha evolucionado manteniendo a MTPROTO como núcleo, pero introduciendo mejoras relevantes en su estructura operativa. En la actualidad, el sistema funciona bajo un esquema de doble capa de cifrado:

Cifrado cliente-servidor, aplicado en los chats en la nube, que permite la sincronización transparente de mensajes en distintos dispositivos (véase Fig.1).

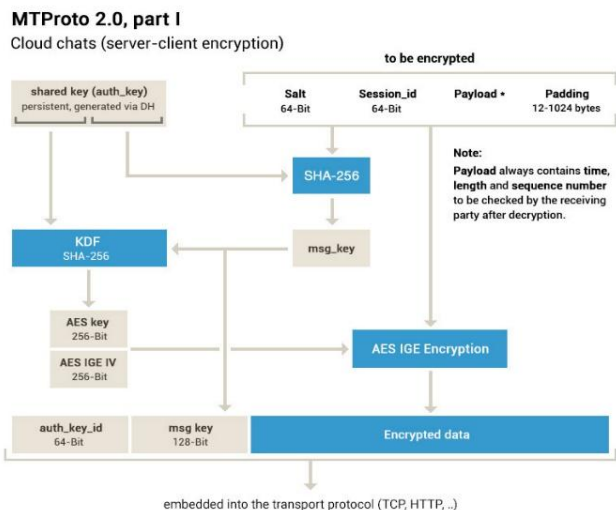


Figura 1. Flujo de Cifrado de MTPROTO 2.0 para Chats en la Nube (Servidor-Cliente)

Cada mensaje de texto simple que se va a cifrar en MTPROTO siempre contiene los siguientes datos:

- server salt
- session id
- payload (time + length + sequence number)
- padding

Cifrado de extremo a extremo, reservado para los chats secretos y las comunicaciones de voz o video, donde las claves de sesión nunca abandonan los dispositivos de los usuarios (véase Fig. 2).

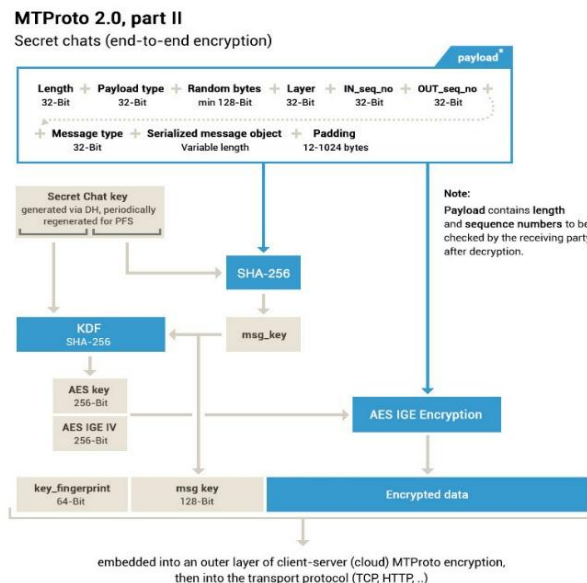


Figura 2. Flujo de Cifrado de MTPROTO 2.0 para Chats Seguros (Cliente-Cliente)

El establecimiento de un chat secreto implica la negociación de parámetros mediante Diffie-Hellman y la generación de una clave compartida de 2048 bits, la cual es renovada de manera periódica (cada 100 mensajes transmitidos o cada semana de uso). Las claves anteriores se eliminan de forma irreversible, dificultando ataques retrospectivos.

Además, Telegram ha incorporado un mecanismo de autenticación visual mediante imágenes que representan la clave, con el propósito de permitir a los usuarios verificar manualmente la ausencia de intermediarios en la comunicación.

En términos evolutivos, Telegram ha reforzado aspectos como la rotación dinámica de claves, la tolerancia a conexiones inestables y la eficiencia en la transferencia de archivos de gran tamaño. Sin embargo, la aplicación continúa enfrentando cuestionamientos por mantener un protocolo propio en lugar de adoptar estándares consolidados como AES-GCM con HMAC o el Signal Protocol, que proporcionan mayores garantías formales de seguridad y han sido ampliamente auditados [3] [4].

C. Problemas y vulnerabilidades

La principal problemática del sistema de cifrado o protección a la información se encuentra en el uso de sus primitivas y el alto nivel de vulnerabilidad que manejan. Esto se debe principalmente a que no satisface las definiciones de cifrado autenticado (AE), que busca cubrir los servicios de integridad y autenticidad, ni con indistinguibilidad bajo ataque de texto cifrado elegido (INDCCA)¹, cuyo servicio principal

¹ La indistinguibilidad bajo ataque de texto cifrado elegido (IND-CCA) significa que, aunque un atacante pueda probar a descifrar muchos mensajes cifrados, no puede adivinar cuál de dos mensajes fue realmente cifrado.

es la confidencialidad frente ataques activos. Por ende podríamos interpretar que el protocolo MTProto no cumple con garantizar la seguridad de los esquemas de cifrado simétrico en presencia de ataques activos o en otras palabras permite que el adversario logre acceder a los textos cifrados y tiene acceso a un oráculo de descifrado[3].

Así mismo el sistema de cifrado común en la mayoría de los chats es exclusivamente cliente servidor permitiendo que los mensajes que no se encuentren en los chats secretos o en esta categoría puedan ser vulnerados. De igual forma se han realizado diversas pruebas de ataque para demostrar que MTProto no es seguro con base en IND-CCA siendo:

1) Ataque #1: Extensión de longitud de Relleno

La idea general de este tipo de ataque se basa en obtener un texto cifrado valido $c = (tag, y_1, \dots, y_l)$ y agregarle al final de este un bloque aleatorio r obteniendo $c' = (tag, y_1, \dots, y_l, r)$.

Al desencriptar c' la carga x' tendrá un pad' más largo, y al este no ser autenticado y no ser una entrada para la función de la etiqueta de autenticación (tag), nunca se detecta esta falla, desencadenando en que el descifrado se complete con éxito y nos revele el mensaje original con una probabilidad del 100%[4].

2) Ataque #2 Sustitución del Último Bloque

Al igual que el ataque anterior obtendremos un texto cifrado y remplazaremos su último bloque por uno completamente aleatorio.

Lo que se busca principalmente con esto es explotar una propiedad del modo de descifrado IGE, esto se debe principalmente a que al decifrar, los ultimos bits del mensaje se distribuyen aleatoriamente debido al uso de una Función Pseudoaleatoria (PRF), y para que se valide y obtengamos el mensaje tendría que surgir la casualidad de que estos bits aleatorios coincidan con los bits del mensaje original, lo que en caso de suceder nos permitiría validar la autenticación y obtener el mensaje original [4].

III. PRIMITIVAS CRIPTOGRAFICAS

A. Criptografía de Clave Simétrica

Esta primitiva se define como una operación donde el emisor y el receptor comparten una única clave para cifrar y descifrar la información.

El protocolo implementa esta función utilizando el algoritmo AES, se usa una clave de 256 bits para cifrar el contenido de los mensajes (payload). La clave específica para cada mensaje se deriva de una clave compartida a largo plazo (auth_key) y se utiliza en un modo de operación llamado IGE.

B. Criptografía de Clave Pública

Esta primitiva se define como una función que utiliza un par de claves (una pública y una privada) para el cifrado y descifrado.

Cuando un usuario se registra en Telegram, crea una clave pública y privada. La clave pública se envía a los servidores

de Telegram, mientras que la clave privada se almacena localmente en el dispositivo del usuario.

C. Función Hash Unidireccional

Esta primitiva se define como una función que convierte una entrada de longitud variable en una salida de longitud fija.

El protocolo utiliza esta primitiva para garantizar la integridad generando una etiqueta de autenticación (tag) que verifica que el mensaje no sea alterado, y autenticidad creando una huella digital con el uso del tag (key_fingerprint) de la clave compartida en los chats secretos.

IV. ALGORITMOS

A. Algoritmos y su proposito

A través del protocolo MTProto, Telegram emplea un gran conjunto de algoritmos criptográficos, los cuales en conjunto combinan cifrado simétrico, asimétrico, funciones hash y mecanismos de derivación de claves, haciendo que cada uno de estos roles cumpla un rol específico en la protección de la información. A continuación se detallaran los algoritmos empleados por MTProto y el funcionamiento que cumplen cada uno de estos para la protección de la información.

- 1) AES-256 en modo IGE (Inifinite Garble Extension): El Advanced Encryption Estándar o mejor conocido como AES, es un cifrador simétrico, adoptado como estándar en 2001 por NIST, teniendo como la variante principal y más robusta a AES-256 por su clave de 256 bits. En Telegram su función es proteger la confidencialidad, esto lo realiza asegurando que solo las partes que tengan la clave correcta puedan descifrar el mensaje y junto con el modo IGE permite cifrar los mensajes de manera que cualquier pequeña modificación en un bloque afectaría toda la secuencia cifrada[5].
- 2) RSA-2048: Es un algoritmo de criptografía asimétrica. Este algoritmo es altamente usado para la autenticación y establecimiento de claves, y el uso en Telegram no es la excepción, esto principalmente a que su trabajo es autorizar y distribuir las claves seguras, asegurándose que los usuarios y servidores se autenticuen mutuamente y que las claves de la sesión sean legítimas.
- 3) Diffie-Hellman (DH Generation): Siendo principalmente un protocolo de intercambio de claves, permite a dos partes acordar un secreto en común a través de un canal inseguro. Su uso en Telegram radica principalmente en establecer una clave compartida entre los diversos dispositivos que participan en un chat secreto sin necesidad de transmitir la clave directamente, siendo a su vez la base principal para poder aplicar AES-256.
- 4) SHA-1: El Secure Hash Algorithm 1 o mejor conocido como SHA-1, genera un has de 160 bits, y aunque hoy en día es considerado como obsoleto por su alto índice de vulnerabilidades, aun es parte MTProto, siendo utilizado ampliamente para garantizar la integridad de los datos, autenticación y firmas digitales.

- 5) SHA-256:
Por su parte el SHA-256, es parte de la familia SHA-2, siendo una mayor version del SHA-1, puesto que se considera seguro en la actualidad y es ampliamente usado en sistemas de integridad, firmas digitales y blockchain. Actualmente en Telegram es utilizado como un derivador de claves de sesión y verificador de la integridad de los components modernos del protocol MTProto.
- 6) KDF (Key Derivation Function):
Es una función diseñada para derivar claves seguras a partir de un secreto maestro. En MTProto se hace uso de un KDF propio basado en multiples invocaciones de SHA-1, permitiendo obtener claves de corto plazo a partir de la clave de sesión larga derivada por DH, asegurando variabilidad y un grado mayor de dificultad para la explotación de claves repetidas.

B. Clasificación Histórica

Todos los algoritmos que componen el protocolo MTProto pertenecen a la era de la **criptografía moderna**.

Dentro de la criptografía moderna, se destacan dos algoritmos fundacionales que sentaron las bases para la comunicación segura en redes abiertas (70's):

- Diffie-Hellman (DH): Propuesto en 1976, introdujo el revolucionario concepto de intercambio de claves sobre un canal inseguro.
- RSA: Publicado en 1977, fue uno de los primeros sistemas de clave pública funcionales para cifrado y firma.

El núcleo del cifrado y la integridad de datos en MTProto se basa en estándares desarrollados para las necesidades de la era de internet (90's y 2000):

- AES-256: Adoptado como estándar en 2001, el Advanced Encryption Standard es el cifrador simétrico de referencia a nivel mundial
- SHA-2 y SHA-1: La familia de algoritmos Secure Hash Algorithm (SHA) es crucial. Sin embargo, SHA-1 (publicado en 1995), un algoritmo que hoy en día se considera obsoleto e inseguro debido a su vulnerabilidad a ataques de colisión.
- KDF: Es un tipo de algoritmo moderno cuya función es derivar una o más claves específicas a partir de un secreto maestro.

V. TIPO DE CRIPTOGRAFIA

La arquitectura de seguridad del protocolo MTProto de Telegram se fundamenta en un modelo híbrido que combina dos tipos de criptografía: simétrica y asimétrica. Cada una cumple una función específica y esencial dentro del flujo de comunicación segura.

A. Criptografía Simétrica para la Confidencialidad

La confidencialidad de los datos en tránsito es garantizada mediante el uso de criptografía simétrica. Este enfoque se emplea para el cifrado de todo el contenido de los mensajes, ya sean textos, archivos multimedia u otros documentos, y es

la base tanto para los chats en la nube (cifrado servidor-cliente) como para los chats secretos (cifrado cliente-cliente).

El protocolo implementa específicamente el algoritmo AES con una longitud de clave de 256 bits. Dicho algoritmo se utiliza en un modo de operación conocido como Infinite Garble Extension para procesar la carga útil (payload) de los mensajes.

B. Criptografía Asimétrica para el Intercambio de Claves

Para que el cifrado simétrico sea seguro, es necesario que la clave compartida se establezca de una manera que impida su interceptación. Esta función es resuelta mediante el uso de criptografía asimétrica. Su propósito principal en MTProto no es cifrar los mensajes, sino facilitar el intercambio seguro de claves.

El protocolo utiliza de manera fundamental el intercambio de claves Diffie-Hellman (DH). Este método permite que dos partes generen una clave secreta compartida. Adicionalmente, el uso de RSA en las etapas iniciales para proteger el primer intercambio de información antes de DH.

VI. MANEJO DE LA INFORMACIÓN

El protocolo MTProto emplea un manejo de información basado en el cifrado por bloques, en contraposición a un cifrado de flujo que procesa los datos de forma continua, bit a bit. Esta elección es una consecuencia directa del uso del algoritmo AES, el cual es, por definición, un cifrador por bloques robusto y ampliamente estandarizado en la industria.

En el caso de la implementación de AES en MTProto, los datos del mensaje se segmentan en bloques de 128 bits (16 bytes) antes de ser procesados por el algoritmo. Cada uno de estos bloques se cifra de forma individual, aunque en el modo de operación IGE utilizado por el protocolo, el cifrado de un bloque depende del bloque de texto plano y del bloque de texto cifrado que le preceden, creando una cadena de dependencias.

VII. CONCLUSIONES

El análisis del protocolo MTProto de Telegram presenta una conclusión dual: si bien su arquitectura de seguridad se fundamenta en primitivas criptográficas empleadas en algoritmos robustos y reconocidos como AES-256 y el intercambio de claves Diffie-Hellman, su implementación en un protocolo de diseño propio, no estándar, introduce vulnerabilidades significativas que comprometen sus garantías de seguridad.

Este caso subraya los riesgos de desarrollar criptografía a medida. La seguridad de un sistema no depende únicamente de la fortaleza de sus algoritmos individuales, sino de la rigurosidad del protocolo que los combina.

REFERENCES

- [1] ["Preguntas frecuentes," Telegram. <https://telegram.org/faq#p-que-tan-seguro-es-telegram>]
- [2] ["Telegram Privacy Policy," Telegram. <https://telegram.org/privacy#4-keeping-your-personal-data-safe>]
- [3] [J. Jakobsen y C. Orlandi, "On the CCA (in)Security of MTProto", en CCS'16: 2016 ACM SIGSAC Conf. Comput. Commun. Secur., Vienna Austria. New York, NY, USA: ACM, 2016. Accedido el 1 de octubre de 2025. [En línea]. Disponible: <https://doi.org/10.1145/2994459.2994468>]

- [4] Binb, "La 'Privacidad' de Telegram: Aclarando qué tan seguro es Telegram realmente : r/cryptography." https://www.reddit.com/r/cryptography/comments/1f8mm0k/telegram_s_privacy_lets_clarify_how_safe_telegram/?tl=es-419
- [5] E. H. Escobar, M. Z. Chaudhary, H. Darwish, A. Sherif, M. Elersy and K. Khalil, "Key-Based Authentication Scheme for Evtol Drones Using Chebyshev Chaotic Maps," 2025 IEEE 4th International Conference on Computing and Machine Intelligence (ICMI), MI, USA, 2025, pp. 1-6, doi: 10.1109/ICMI65310.2025.11141241.
- [6] «FAQ for the Technically Inclined», *Telegram*. <https://core.telegram.org/techfaq#q-why-did-you-go-for-a-custom-protocol>
- [7] F. Martín-Fernández y P. Caballero-Gil, "Analysis of the New Standard Hash Function," *arXiv preprint* arXiv:2209.11857, 2022.
- [8] S. Bäuml, C. Pascual-García, V. Wright, O. Fawzi y A. Acín, "Security of discrete-modulated continuous-variable quantum key distribution," *arXiv preprint* arXiv:2303.09255, 2023.
- [9] Y. Yu, "Encryption Technology for Computer Network Data Security Protection," *Security and Communication Networks*, vol. 2022, Art. ID 1789222, 9 pp., 2022. doi: 10.1155/2022/1789222.
- [10] M. Seo, J. H. Park, Y. Kim, S. Cho, D. H. Lee, y J. Y. Hwang, "Construction of a New Biometric-Based Key Derivation Function and Its Application," *Security and Communication Networks*, vol. 2018, Art. ID 6107912, 14 pp., 2018. doi: 10.1155/2018/6107912.